

Section 98 Editorial — Plug DP And Treewidth Frontiers

A. Tower Domino Tilings

How to find it

Build the one-row mask transition matrix and exponentiate it.

Correctness

Matrix multiplication composes consecutive row transitions; binary exponentiation therefore represents exactly H rows.

Complexity

$O(2^{(3W)} \log H)$ with dense matrices.

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll=long long;
const int MOD=1e9+7;
using Mat=vector<vector<int>>>;
Mat mul(const Mat&a,const Mat&b) {
    int n=a.size();
    Mat c(n,vector<int>(n));
    for(int i=0;i<n;i++)for(int k=0;k<n;k++)if(a[i][k])for(int j=0;j<n;j++)c[i][j]=(c[i][j]+(ll)a[i][k]*b[k][j])%MOD;
    return c;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    unsigned long long h;
    int w;
    cin>>h>>w;
    int n=1<<w;
    Mat a(n,vector<int>(n));
    for(int in=0;in<n;in++) {
        function<void(int,int,int)>fill=[&](int c,int used,int out) {
            if(c==w) {
                a[in][out]++;
                return;
            }
            int bit=1<<c;
            if(used&bit)fill(c+1,used,out);
            else {
                if(c+1<w&&!(used&(bit<<1)))fill(c+2,used|bit|(bit<<1),out);
                fill(c+1,used|bit,out|bit);
            }
        };
        fill(0,in,0);
    }
}
```

```

}
Mat r(n,vector<int>(n));
for(int i=0;i<n;i++)r[i][i]=1;
while(h) {
    if(h&1)r=mul(r,a);
    a=mul(a,a);
    h>>=1;
}
cout<<r[0][0]<<'\\n';
}

```

Python

```

import sys
MOD = 1000000007

def mul(a, b):
    n = len(a)
    c = [[0] * n for _ in range(n)]
    for i in range(n):
        for k, x in enumerate(a[i]):
            if x:
                for j, y in enumerate(b[k]):
                    c[i][j] = (c[i][j] + x * y) % MOD
    return c

def main():
    h, w = map(int, sys.stdin.buffer.read().split())
    size = 1 << w
    mat = [[0] * size for _ in range(size)]
    for incoming in range(size):

        def fill(c, used, out):
            if c == w:
                mat[incoming][out] += 1
                return
            bit = 1 << c
            if used & bit:
                fill(c + 1, used, out)
            else:
                if c + 1 < w and (not used & bit << 1):
                    fill(c + 2, used | bit | bit << 1, out)
                fill(c + 1, used | bit, out | bit)
        fill(0, incoming, 0)
    result = [[int(i == j) for j in range(size)] for i in range(size)]
    while h:
        if h & 1:
            result = mul(result, mat)
            mat = mul(mat, mat)
            h //= 2
    print(result[0][0])
if __name__ == '__main__':
    main()

```

B. Connected Cell Sets

How to find it

Store canonical labels for open frontier components and reject premature component disappearance.

Correctness

Labels encode exactly future-connectable components. A vanished component cannot meet future cells; the closure rule is therefore necessary and sufficient for one final component.

Complexity

Exponential in width and polynomial in height.

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll=long long;
const int MOD=1e9+7;
using State=pair<vector<int>,bool>;
vector<int> canon(vector<int>a) {
    map<int,int>mp;
    int nxt=1;
    for(int&x:a)if(x) {
        if(!mp.count(x))mp[x]=nxt++;
        x=mp[x];
    }
    return a;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int h,w;
    cin>>h>>w;
    vector<string>g(h);
    for(auto&s:g)cin>>s;
    map<State,int>dp;
    dp[ {
        vector<int>(w),false
    }
    ]=1;
    for(int r=0;r<h;r++)for(int c=0;c<w;c++) {
        map<State,int>nd;
        for(auto[state,value]:dp) {
            auto labels=state.first;
            bool closed=state.second;
            int up=labels[c],left=c?labels[c-1]:0;
            auto z=labels;
            z[c]=0;
            bool vanished=up&&find(z.begin(),z.end(),up)==z.end();
            if(!vanished||(!closed&&count_if(z.begin(),z.end(),[](int x){return x;})==0)) {
                State key= {
                    canon(z),closed||vanished
                };
                ;
                nd[key]=(nd[key]+value)%MOD;
            }
            if(g[r][c]=='.'&&!closed) {
                z=labels;
                if(left&&up&&left!=up)for(int&x:z)if(x==up)x=left;
                z[c]=left?left:(up?up:*max_element(z.begin(),z.end())+1);
                State key= {
                    canon(z),false
                };
                ;
            }
        }
    }
```

```

        nd[key]=(nd[key]+value)%MOD;
    }
}
dp.swap(nd);
}
ll answer=0;
for(auto [state,value]:dp) {
    set<int>s(state.first.begin(),state.first.end());
    s.erase(0);
    if(state.second||s.size()==1)answer+=value;
}
cout<<answer%MOD<<'\\n';
}

```

Python

```

import sys
MOD = 1000000007

def canon(a):
    mp = {}
    nxt = 1
    out = []
    for x in a:
        if x and x not in mp:
            mp[x] = nxt
            nxt += 1
        out.append(mp.get(x, 0))
    return tuple(out)

def main():
    data = sys.stdin.buffer.read().split()
    h, w = map(int, data[:2])
    g = [x.decode() for x in data[2:]]
    dp = {(tuple([0] * w), False): 1}
    for r in range(h):
        for c in range(w):
            nd = {}
            for (labels, closed), value in dp.items():
                old = list(labels)
                up = old[c]
                left = old[c - 1] if c else 0
                new = old[:]
                new[c] = 0
                vanished = up and up not in new
                if not vanished or (not any(new) and (not closed)):
                    key = (canon(new), closed or bool(vanished))
                    nd[key] = (nd.get(key, 0) + value) % MOD
            if g[r][c] == '.' and (not closed):
                new = old[:]
                if left and up and (left != up):
                    new = [left if x == up else x for x in new]
                new[c] = left or up or max(new, default=0) + 1
                key = (canon(new), False)
                nd[key] = (nd.get(key, 0) + value) % MOD
            dp = nd
    answer = 0
    for (labels, closed), value in dp.items():
        if closed or len(set(labels) - {0}) == 1:
            answer = (answer + value) % MOD
    print(answer)

```

```
if __name__ == '__main__':
    main()
```

C. Terminal Steiner Network

How to find it

Merge terminal submasks at a vertex, then relax locations with multi-source Dijkstra.

Correctness

Every transition joins connected structures, and splitting an optimum at its last merge yields a represented transition.

Complexity

$O(3^k n + 2^k m \log n)$.

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll=long long;
const ll INF=4e18;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n,m,k;
    cin>>n>>m>>k;
    vector<vector<pair<int,int>>>g(n);
    while(m--) {
        int u,v,w;
        cin>>u>>v>>w;
        --u;
        --v;
        g[u].push_back({v,w});
        g[v].push_back({u,w});
    }
    vector<vector<ll>>dp(1<<k,vector<ll>(n,INF));
    for(int i=0,v;i<k;i++)cin>>v,dp[1<<i][v-1]=0;
    for(int mask=1;mask<1<<k;mask++) {
        for(int sub=(mask-1)&mask;sub;sub=(sub-1)&mask) {
            int other=mask^sub;
            if(sub<other)for(int v=0;v<n;v++)dp[mask][v]=min(dp[mask][v],dp[sub][v]+dp[other][v]);
        }
        priority_queue<pair<ll,int>,vector<pair<ll,int>>,greater<pair<ll,int>>>q;
        for(int v=0;v<n;v++)if(dp[mask][v]<INF)q.push({dp[mask][v],v});
        while(!q.empty()) {
            auto[d,u]=q.top();
            q.pop();
            if(d!=dp[mask][u])continue;
            for(auto[v,w]:g[u])if(d+w<dp[mask][v])dp[mask][v]=d+w,q.push({d+w,v});
        }
    }
    ll ans=*min_element(dp.back().begin(),dp.back().end());
    if(ans==INF)cout<<"IMPOSSIBLE\n";
    else cout<<ans<<'\\n';
}
```

Python

```
import sys, heapq

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    n, m, k = d[:3]
    at = 3
    g = [[] for _ in range(n)]
    for _ in range(m):
        u, v, w = d[at:at + 3]
        at += 3
        u -= 1
        v -= 1
        g[u].append((v, w))
        g[v].append((u, w))
    terminals = [x - 1 for x in d[at:at + k]]
    inf = 10 ** 30
    dp = [[inf] * n for _ in range(1 << k)]
    for i, v in enumerate(terminals):
        dp[1 << i][v] = 0
    for mask in range(1, 1 << k):
        sub = mask - 1 & mask
        while sub:
            other = mask ^ sub
            if sub < other:
                for v in range(n):
                    dp[mask][v] = min(dp[mask][v], dp[sub][v] + dp[other][v])
            sub = sub - 1 & mask
        q = [(x, v) for v, x in enumerate(dp[mask]) if x < inf]
        heapq.heapify(q)
        while q:
            cost, u = heapq.heappop(q)
            if cost != dp[mask][u]:
                continue
            for v, w in g[u]:
                if cost + w < dp[mask][v]:
                    dp[mask][v] = cost + w
                    heapq.heappush(q, (cost + w, v))
        ans = min(dp[-1])
        print('IMPOSSIBLE' if ans == inf else ans)
if __name__ == '__main__':
    main()
```

D. Nice-Decomposition Independent Set

How to find it

DP over selected active-bag vertices through leaf, introduce, forget, and join nodes.

Correctness

Introduce checks all newly visible edges; forget is safe by decomposition connectivity; join combines independent subtrees and subtracts duplicated bag weights.

Complexity

$O(t * 2^w * w)$ expected map work.

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll=long long;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n,m;
    cin>>n>>m;
    vector<ll>w(n);
    for(ll&x:w)cin>>x;
    vector<unsigned long long>adj(n);
    while(m--) {
        int u,v;
        cin>>u>>v;
        --u;
        --v;
        adj[u] |= 1ULL<<v;
        adj[v] |= 1ULL<<u;
    }
    int t;
    cin>>t;
    vector<unsigned long long>bag(t);
    vector<map<unsigned long long,ll>>dp(t);
    for(int i=0;i<t;i++) {
        char type;
        cin>>type;
        if(type=='L')dp[i][0]=0;
        else if(type=='I') {
            int child,v;
            cin>>child>>v;
            --child;
            --v;
            bag[i]=bag[child] | 1ULL<<v;
            dp[i]=dp[child];
            for(auto [mask,val]:dp[child])if(!(mask&adj[v])) {
                auto nextmask=mask | 1ULL<<v;
                ll candidate=val+w[v];
                if(!dp[i].count(nextmask) || candidate>dp[i][nextmask])dp[i][nextmask]=candidate;
            }
        }
        else if(type=='F') {
            int child,v;
            cin>>child>>v;
            --child;
            --v;
            bag[i]=bag[child]&~(1ULL<<v);
            for(auto [mask,val]:dp[child]) {
                auto nextmask=mask&~(1ULL<<v);
                if(!dp[i].count(nextmask) || val>dp[i][nextmask])dp[i][nextmask]=val;
            }
        }
        else {
            int a,b;
            cin>>a>>b;
            --a;
            --b;
            bag[i]=bag[a];
            for(auto [mask,val]:dp[a])if(dp[b].count(mask)) {
                ll duplicate=0;
                for(int v=0;v<n;v++)if(mask>>v&1)duplicate+=w[v];
                dp[i][mask]=val+dp[b][mask]-duplicate;
            }
        }
    }
}
```

```

    }
}
cout<<dp.back()[0]<<'\n';
}

```

Python

```

import sys

def main():
    lines = sys.stdin.buffer.readlines()
    n, m = map(int, lines[0].split())
    weight = list(map(int, lines[1].split()))
    adj = [0] * n
    for line in lines[2:2 + m]:
        u, v = map(int, line.split())
        u -= 1
        v -= 1
        adj[u] |= 1 << v
        adj[v] |= 1 << u
    t = int(lines[2 + m])
    dp = []
    bags = []
    for line in lines[3 + m:3 + m + t]:
        z = line.split()
        kind = z[0]
        if kind == b'L':
            bags.append(0)
            dp.append({0: 0})
        elif kind == b'I':
            child = int(z[1]) - 1
            v = int(z[2]) - 1
            bags.append(bags[child] | 1 << v)
            cur = dict(dp[child])
            for mask, val in dp[child].items():
                if not mask & adj[v]:
                    cur[mask | 1 << v] = max(cur.get(mask | 1 << v, -10 ** 30), val + weight[v])
            dp.append(cur)
        elif kind == b'F':
            child = int(z[1]) - 1
            v = int(z[2]) - 1
            bags.append(bags[child] & ~(1 << v))
            cur = {}
            for mask, val in dp[child].items():
                key = mask & ~(1 << v)
                cur[key] = max(cur.get(key, -10 ** 30), val)
            dp.append(cur)
        else:
            a = int(z[1]) - 1
            b = int(z[2]) - 1
            bags.append(bags[a])
            cur = {}
            for mask, val in dp[a].items():
                if mask in dp[b]:
                    cur[mask] = val + dp[b][mask] - sum((weight[v] for v in range(n) if mask >> v & 1))
            dp.append(cur)
    print(dp[-1].get(0, max(dp[-1].values())))
if __name__ == '__main__':
    main()

```

Forgetting proof and stress companion

When the last frontier occurrence of a component label disappears, that component has no interface with unseen cells and can never reconnect. Reject the transition unless it completes the sole connected object. Canonicalize labels after every merge and deletion so state identity depends on the partition, not historical names.

At tree-decomposition joins, children overlap exactly on the bag; require compatible bag assignments and correct contributions counted in both children. Validate the decomposition itself before debugging the DP. Compare all four problems with exhaustive small instances, including an empty terminal set, single terminals, components closed too early, zero repeated rows, and every nice-node type at the root boundary.